Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

Практична робота №1

з дисципліни «Розробка проектів засобами платформи .NET»

Тема: «Базові конструкції C#. Консольний застосунок»

Виконав: студент гр. 2ПІ-24б
Слободян І. В.

Перевірив:
Позур Михайло Юрійович

м. Вінниця - 2026

GitHub репозиторій: https://github.com/2005igor2023-crypto/dotnetvntu

1. Мета роботи

Закріпити навички роботи з базовими типами даних, операторами, керуючими конструкціями (розгалуження, цикли), масивами та методами мови C#; навчитися організовувати консольний додаток з введенням/виведенням даних та структурувати найпростіший застосунок у вигляді меню з пунктами.

2. Завдання варіанта №1 (№17 за списком)

• **Предметна область:** Статистика результатів тестування.

• **Вхідні дані:** Масив цілих чисел int[] scores - бали студентів за одне тестування (діапазон 0-100), без прив'язки до імен. Початковий масив заповнений тестовими даними наперед.

• **Вимоги до функціоналу (меню):**

0. Вихід з програми.

1. Додати результат - розширення масиву за допомогою Array.Resize(ref scores, scores.Length + 1); введення числа від 0 до 100 з валідацією через int.TryParse.

2. Показати всі результати - форматований вивід усіх збережених балів.

3. Показати середній бал - метод CalculateAverage(int[] scores) повертає дробове число double з округленням до 2 знаків після коми (:F2).

4. Показати найвищий і найнижчий бал - методи FindMaxIndex та FindMinIndex знаходять екстремуми та їхні позиції (індекси) у масиві.

5. Показати розподіл за категоріями:

• «відмінно» (90-100 балів);

• «добре» (75-89 балів);

• «задовільно» (60-74 бали);

• «незадовільно» (<60 балів).

Визначення категорії реалізовано за допомогою switch expression.

6. Додатково: Генерація випадкових оцінок для тестування великих вибірок (Random.Shared).

7. Додатково: Очищення масиву для перевірки граничного випадку.

• **Граничний випадок:** Масив з 0 результатів (порожній масив). Програма не викликає виняткових ситуацій (наприклад, ділення на 0 при обчисленні середнього або IndexOutOfRangeException), а виводить зрозуміле повідомлення «Немає даних» для всіх аналітичних операцій.

3. Короткий опис реалізації та архітектури програми

1. **Структура проєкту:** Консольний застосунок (.NET 10, C# 13/latest), оформлений у традиційному стилі з класом Program та точкою входу static void Main() (без top-level statements).

2. **Кодування консолі:** На початку Main встановлено Console.OutputEncoding = Encoding.UTF8; та Console.InputEncoding = Encoding.UTF8; для бездоганного відображення української мови.

3. **Керування меню:** Головний цикл while (isRunning) використовує switch expression для маршрутизації дій за обраним пунктом.

4. **Валідація введення:** Метод ReadInt(string prompt, int min, int max) реалізує циклічний запит із використанням int.TryParse(). Будь-яке некоректне введення (символи, текст, числа поза діапазоном) безпечно відхиляється без аварійного завершення програми.

5. **Робота з пам'яттю:** Масив розширюється «на місці» через передачу за посиланням ref int[] scores та виклик Array.Resize.

4. Лістинг програми (`Program.cs`)

```
using System;
using System.Text;

namespace CSharpBasics;

internal class Program
{
    private static void Main()
    {
        Console.OutputEncoding = Encoding.UTF8;
```

```
        Console.InputEncoding = Encoding.UTF8;

        int[] scores = { 85, 92, 58, 74, 90, 63, 45, 99, 78, 88 };
        bool isRunning = true;

        while (isRunning)
        {
            PrintMenu();
            int choice = ReadInt("Ваш вибір: ", 0, 7);
            Console.WriteLine();

            isRunning = choice switch
            {
                1 => HandleAddScore(ref scores),
                2 => HandleShowAllScores(scores),
                3 => HandleShowAverage(scores),
                4 => HandleShowMinMax(scores),
                5 => HandleShowCategoryDistribution(scores),
                6 => HandleGenerateRandomScores(ref scores),
                7 => HandleClearScores(ref scores),
                0 => false,
                _ => true
            };

            if (isRunning)
            {
                Console.WriteLine("\nНатисніть Enter для повернення в меню...");
                Console.ReadLine();
            }
        }

        Console.WriteLine("Програму успішно завершено. Гарного дня!");
    }

    private static void PrintMenu()
    {

Console.WriteLine("\n=========================================================");
        Console.WriteLine("    Практична робота №1 | Варіант 1 (№17 у списку)   ");
        Console.WriteLine("          Статистика результатів тестування          ");

Console.WriteLine("=========================================================");
        Console.WriteLine("1. Додати результат тестування (бал від 0 до 100)");
        Console.WriteLine("2. Показати всі результати");
        Console.WriteLine("3. Показати середній бал");
        Console.WriteLine("4. Показати найвищий і найнижчий бал (з позиціями)");
        Console.WriteLine("5. Показати розподіл за категоріями");
        Console.WriteLine("6. Згенерувати випадкові бали (тестовий набір)");
        Console.WriteLine("7. Очистити всі результати (перевірка порожнього масиву)");
        Console.WriteLine("0. Вихід");
        Console.WriteLine("---------------------------------------------------------");
    }

    private static bool HandleAddScore(ref int[] scores)
    {
        Console.WriteLine("--- Додавання нового результату ---");
        int newScore = ReadInt("Введіть бал студента (від 0 до 100): ", 0, 100);
```

```
        AddScore(ref scores, newScore);
        Console.WriteLine($"\n[Успішно] Результат {newScore} додано до списку (поточна кількість: {scores.Length}).");
        return true;
    }

    private static bool HandleShowAllScores(int[] scores)
    {
        Console.WriteLine("--- Список усіх результатів ---");
        if (scores.Length == 0)
        {
            Console.WriteLine("Немає даних. Масив результатів порожній.");
            return true;
        }

        Console.WriteLine($"Усього результатів: {scores.Length}");
        Console.Write("Бали: ");
        for (int i = 0; i < scores.Length; i++)
        {
            Console.Write($"[{i + 1}] {scores[i]}");
            if (i < scores.Length - 1) Console.Write(", ");
        }
        Console.WriteLine();
        return true;
    }

    private static bool HandleShowAverage(int[] scores)
    {
        Console.WriteLine("--- Середній бал тестування ---");
        if (scores.Length == 0)
        {
            Console.WriteLine("Немає даних. Неможливо обчислити середній бал для порожнього масиву.");
            return true;
        }

        double average = CalculateAverage(scores);
        Console.WriteLine($"Кількість робіт: {scores.Length}");
        Console.WriteLine($"Середній бал: {average:F2} із 100 можливих.");
        return true;
    }

    private static bool HandleShowMinMax(int[] scores)
    {
        Console.WriteLine("--- Найвищий та найнижчий результат ---");
        if (scores.Length == 0)
        {
            Console.WriteLine("Немає даних. Масив результатів порожній.");
            return true;
        }

        int maxIndex = FindMaxIndex(scores);
        int minIndex = FindMinIndex(scores);

        Console.WriteLine($"Найвищий бал: {scores[maxIndex]} (позиція №{maxIndex + 1}, індекс [{maxIndex}])");
        Console.WriteLine($"Найнижчий бал: {scores[minIndex]} (позиція №{minIndex + 1}, індекс
```

```
[{minIndex}])");
        return true;
    }

    private static bool HandleShowCategoryDistribution(int[] scores)
    {
        Console.WriteLine("--- Розподіл результатів за академічними категоріями ---");
        if (scores.Length == 0)
        {
            Console.WriteLine("Немає даних. Масив результатів порожній.");
            return true;
        }

        int excellent = 0, good = 0, satisfactory = 0, unsatisfactory = 0;
        foreach (int score in scores)
        {
            switch (GetScoreCategory(score))
            {
                case "відмінно": excellent++; break;
                case "добре": good++; break;
                case "задовільно": satisfactory++; break;
                case "незадовільно": unsatisfactory++; break;
            }
        }

        int total = scores.Length;
        Console.WriteLine($"• Відмінно    (90-100 балів): {excellent,3} ст. ({(double)excellent / total *
100:F1}%)");
        Console.WriteLine($"• Добре       (75-89  балів): {good,3} ст. ({(double)good / total *
100:F1}%)");
        Console.WriteLine($"• Задовільно  (60-74  бали ): {satisfactory,3} ст. ({(double)satisfactory /
total * 100:F1}%)");
        Console.WriteLine($"• Незадовільно (<60   балів): {unsatisfactory,3} ст.
({(double)unsatisfactory / total * 100:F1}%)");
        Console.WriteLine("-------------------------------------------------");
        Console.WriteLine($"Разом перевірено робіт: {total}");
        return true;
    }

    private static bool HandleGenerateRandomScores(ref int[] scores)
    {
        Console.WriteLine("--- Генерація випадкового набору балів ---");
        int count = ReadInt("Введіть бажану кількість студентів (від 1 до 100): ", 1, 100);
        scores = new int[count];
        Random rnd = Random.Shared;
        for (int i = 0; i < count; i++) scores[i] = rnd.Next(0, 101);
        Console.WriteLine($"\n[Успішно] Згенеровано {count} випадкових результатів.");
        return true;
    }

    private static bool HandleClearScores(ref int[] scores)
    {
        Console.WriteLine("--- Очищення результатів ---");
        scores = Array.Empty<int>();
        Console.WriteLine("[Успішно] Масив результатів очищено. Тепер він порожній (0
елементів).");
        return true;
```

```
    }

    public static void AddScore(ref int[] scores, int newScore)
    {
        Array.Resize(ref scores, scores.Length + 1);
        scores[^1] = newScore;
    }

    public static double CalculateAverage(int[] scores)
    {
        if (scores.Length == 0) return 0.0;
        long sum = 0;
        foreach (int score in scores) sum += score;
        return (double)sum / scores.Length;
    }

    public static int FindMaxIndex(int[] scores)
    {
        if (scores.Length == 0) return -1;
        int maxIndex = 0;
        for (int i = 1; i < scores.Length; i++)
        {
            if (scores[i] > scores[maxIndex]) maxIndex = i;
        }
        return maxIndex;
    }

    public static int FindMinIndex(int[] scores)
    {
        if (scores.Length == 0) return -1;
        int minIndex = 0;
        for (int i = 1; i < scores.Length; i++)
        {
            if (scores[i] < scores[minIndex]) minIndex = i;
        }
        return minIndex;
    }

    public static string GetScoreCategory(int score) => score switch
    {
        >= 90 => "відмінно",
        >= 75 => "добре",
        >= 60 => "задовільно",
        _ => "незадовільно"
    };

    public static int ReadInt(string prompt, int min = int.MinValue, int max = int.MaxValue)
    {
        while (true)
        {
            Console.Write(prompt);
            string? input = Console.ReadLine();
            if (int.TryParse(input, out int value) && value >= min && value <= max) return value;
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"Помилка: введіть коректне ціле число в межах від {min} до
{max}.");
            Console.ResetColor();
```

```
        }
    }
}
```

5. Приклади роботи програми (Консольний вивід)

5.1. Показ початкових даних та обчислення середнього бала

```
==========================================================
   Практична робота №1 | Варіант 1 (№17 у списку)
        Статистика результатів тестування
==========================================================
1. Додати результат тестування (бал від 0 до 100)
2. Показати всі результати
3. Показати середній бал
4. Показати найвищий і найнижчий бал (з позиціями)
5. Показати розподіл за категоріями
6. Згенерувати випадкові бали (тестовий набір)
7. Очистити всі результати (перевірка порожнього масиву)
0. Вихід
----------------------------------------------------------
Ваш вибір: 2

--- Список усіх результатів ---
Усього результатів: 10
Бали: [1] 85, [2] 92, [3] 58, [4] 74, [5] 90, [6] 63, [7] 45, [8] 99, [9] 78, [10] 88

Ваш вибір: 3

--- Середній бал тестування ---
Кількість робіт: 10
Середній бал: 77.20 із 100 можливих.
```

5.2. Визначення екстремумів та розподілу за категоріями

```
Ваш вибір: 4

--- Найвищий та найнижчий результат ---
Найвищий бал: 99 (позиція №8, індекс [7])
Найнижчий бал: 45 (позиція №7, індекс [6])

Ваш вибір: 5

--- Розподіл результатів за академічними категоріями ---
• Відмінно    (90-100 балів):   3 ст. (30.0%)
• Добре       (75-89  балів):   3 ст. (30.0%)
• Задовільно  (60-74  бали ):   2 ст. (20.0%)
• Незадовільно (<60   балів):   2 ст. (20.0%)
--------------------------------------------------
Разом перевірено робіт: 10
```

5.3. Перевірка граничного випадку (порожній масив)

```
Ваш вибір: 7

--- Очищення результатів ---
[Успішно] Масив результатів очищено. Тепер він порожній (0 елементів).
```

```
Ваш вибір: 3

--- Середній бал тестування ---
Немає даних. Неможливо обчислити середній бал для порожнього масиву.
```

6. Посилання на GitHub-репозиторій

• **URL репозиторію:** https://github.com/2005igor2023-crypto/dotnetvntu

• **Папка проєкту:** /01-csharp-basics

7. Відповіді на контрольні питання

1. Чим відрізняються value type та reference type в C#? Наведіть приклади.

Відповідь: Value types (значеннєві типи) зберігають безпосередньо свої дані (зазвичай у стеку або як частина іншого об'єкта) та при присвоєнні/передачі копіюються. Приклади: int, double, bool, char, struct. Reference types (посилальні типи) зберігають адресу (посилання) на область пам'яті в керованій купі (managed heap), де розташовані фактичні дані. Приклади: string, object, масиви int[], class.

2. У чому різниця між switch statement та switch expression?

Відповідь: switch statement - це керуюча інструкція, яка виконує блоки коду (секції case: ... break;), не повертаючи значення безпосередньо. switch expression (з'явився в C# 8.0) - це компактний вираз, що одразу повертає обчислене значення через синтаксис =>, не вимагає ключових слів case і break, та перевіряється компілятором на повноту покриття (exhaustiveness).

3. Як коректно обробити некоректне введення користувача без аварійного завершення програми?

Відповідь: Слід використовувати методи безпечного парсингу виду T.TryParse(string?, out T result) (наприклад, int.TryParse), розміщені в циклі while. Якщо метод повертає false або значення поза межами діапазону, виводиться повідомлення про помилку та запит повторюється.

4. Які переваги виокремлення логіки в окремі методи порівняно з кодом в Main?

Відповідь: Дотримання принципу єдиного обов'язку (SRP), повторне використання коду (DRY), спрощення тестування та налагодження, покращення читабельності програми, а також ізоляція областей видимості змінних.

5. Як передаються масиви у методи - за значенням чи за посиланням?

Відповідь: Змінна масиву містить посилання (reference type). За замовчуванням це посилання передається *за значенням* (копіюється адреса). Метод може модифікувати елементи масиву, але не може змінити сам масив (підмінити екземпляр у змінній зухвалого коду). Якщо ж масив передається з модифікатором ref, передається посилання на змінну, що дозволяє замінити масив новим екземпляром (як у Array.Resize).

6. Чим `List<T>` відрізняється від масиву і коли варто обрати кожен з них?

Відповідь: Масив T[] має фіксований розмір після ініціалізації та забезпечує мінімальні накладні витрати на пам'ять. List<T> - це динамічна колекція на базі внутрішнього масиву, яка автоматично збільшує свою місткість при додаванні елементів (Add()). Масив обирають, коли розмір відомий заздалегідь або критична продуктивність; List<T> - коли розмір часто змінюється.

7. У чому різниця між `ref` та `out` параметрами? Наведіть приклад, де підійде лише один з них.

Відповідь: Параметр ref вимагає обов'язкової ініціалізації змінної *до* виклику методу, оскільки метод може як читати, так і записувати її. Параметр out не вимагає ініціалізації перед викликом, але метод *зобов'язаний* присвоїти йому значення до виходу. Приклад: для TryParse потрібен саме out, оскільки змінна результату передається неініціалізованою.

8. Чому рядки (`string`) в C# вважаються незмінними (immutable) і як це впливає на роботу методів `Trim`/`Split`/`Replace`?

Відповідь: Після створення рядок у купі не може бути модифікований. Будь-який метод маніпуляції рядками (Trim, Split, Replace) створює і повертає *новий* об'єкт рядка. Це забезпечує потокобезпечність і можливість інтернування рядків, але для масивних конкатенацій вимагає використання StringBuilder.

9. Коли варто використовувати `var`, а коли - явно вказувати тип змінної?

Відповідь: var доречно використовувати, коли тип змінної однозначно зрозумілий із правої частини виразу (наприклад, var list = new List<int>() або при довгих узагальнених типах). Явний тип варто вказувати для числових літералів (double x = 10;), інтерфейсних типів або коли тип з правого виразу не очевидний.

10. Чим цикл `for` відрізняється від `while` і `foreach`? Наведіть приклад задачі, де кожен із них буде найзручнішим.

Відповідь: for використовує лічильник та найзручніший, коли потрібен індекс елемента або фіксована кількість кроків (пошук індексу екстремуму); foreach перебирає елементи через інтерфейс IEnumerable без використання індексу (підрахунок категорій); while виконується доти, доки умова істинна, та зручний для очікування валідного вводу або організації головного меню.

11. Що станеться під час виконання, якщо не перевірити порожній масив перед обчисленням середнього значення? Як цього уникнути?

Відповідь: При діленні цілих чисел на нуль виникає виняток DivideByZeroException. При діленні дробових чисел double на нуль результат стає NaN (Not a Number), що спотворює подальші обчислення. Уникнути цього можна перевіркою if (scores.Length == 0) return 0.0;.

12. Чим `int.Parse` відрізняється від `int.TryParse`? Чому для введення з клавіатури обирають саме `TryParse`?

Відповідь: int.Parse викидає винятки FormatException або OverflowException при некоректних даних, що вимагає дорогих блоків try-catch. int.TryParse повертає булеве значення false без викидання винятків, забезпечуючи високу швидкодію та чистий код валідації.

8. Висновки

У ході виконання практичної роботи №1 було створено повнофункціональний консольний застосунок мовою C# на платформі .NET 10 для обліку та статистичного аналізу результатів тестування студентів. Було закріплено практичні навички роботи з одновимірними масивами, методами зміни їхнього розміру (Array.Resize), керуючими конструкціями switch expression, а також безпечною валідацією користувацького вводу через int.TryParse. Реалізована архітектура надійно обробляє граничні ситуації (зокрема порожній масив та введення некоректних даних) без аварійних зупинок застосунку.